



RAPPORT TECHNIQUE

SYSTEME D'ANALYSE AUTOMATISEE DE LA CONFORMITE DES CARTONS PAR APPRENTISSAGE PROFOND



2024-2025

REALISE PAR : DAOUKI MAROUANE, ALOUANI IMANE ET DOUAA BOUSSNITRA

Encadré par : Prof Mr. Benlahmar El Habib ET Prof Mme Hanouni Salma

REMERCIEMENTS

Nous souhaitons exprimer nos sincères remerciements à toutes les personnes qui ont contribué à la réalisation de ce projet de système d'analyse automatisée de la conformité des cartons par apprentissage profond.

Tout d'abord, nous adressons nos remerciements les plus chaleureux à nos encadrants :

Professeur Mr. Benlahmar El Habib et **Professeure Mme Hanouni Salma**, pour leur encadrement rigoureux, leurs conseils avisés et leur disponibilité constante. Leur expertise dans le domaine de l'intelligence artificielle et de la vision par ordinateur nous a été d'une aide précieuse pour mener à bien ce travail. Leurs orientations méthodologiques et leurs remarques constructives ont grandement contribué à la qualité de ce projet.

Enfin, nous exprimons notre reconnaissance envers tous ceux qui, de près ou de loin, ont apporté leur soutien à la réalisation de ce projet, notamment par leurs encouragements et leur compréhension durant les phases intensives de développement et de rédaction.

Ce travail n'aurait pas pu aboutir sans cette collaboration et ce soutien collectif.

Résumé

Ce rapport détaille le développement d'un système automatisé d'Analyse de la Conformité des Cartons, conçu pour améliorer l'efficacité de la gestion d'entrepôt. Le projet s'appuie sur des techniques de vision par ordinateur et d'apprentissage profond, notamment le modèle de détection d'objets YOLOv8, pour identifier et analyser les cartons empilés dans des images. Le système comprend un pipeline complet, incluant le prétraitement du jeu de données (redimensionnement, conversion de format, nettoyage des annotations), l'entraînement du modèle, et le développement d'une application Streamlit conviviale. L'application offre des fonctionnalités pour la détection de cartons, l'analyse des surplombs entre paires, l'évaluation de la stabilité des piles, le comptage par zone, et un score global de conformité. Le modèle entraîné a obtenu des résultats prometteurs, avec une précision de 0.954, un rappel de 0.936, et un mAP@50 de 0.975, démontrant sa capacité à détecter avec précision les cartons. L'outil développé vise à rationaliser les processus d'audit, à réduire l'effort manuel, et à améliorer le respect des réglementations d'empilage des cartons.

1. Introduction

1.1. Problématique

Une gestion efficace des entrepôts est cruciale pour les opérations de la chaîne d'approvisionnement. Un aspect clé consiste à s'assurer que les cartons sont empilés et stockés conformément à des règles de conformité spécifiques pour prévenir les dommages, optimiser l'espace et garantir la sécurité. L'inspection manuelle de la conformité prend du temps, est sujette à l'erreur humaine et peut constituer un goulot d'étranglement dans les environnements à rythme rapide. Il existe un besoin pour un système automatisé capable d'analyser les agencements de cartons et de signaler les empilements non conformes.

1.2. Objectifs du Projet

Les objectifs principaux de ce projet étaient :

- Développer un modèle de détection d'objets précis capable d'identifier les cartons individuels dans diverses configurations d'empilement.
- Mettre en œuvre un pipeline de prétraitement pour le jeu de données SCD (Stacked Carton Dataset), incluant le redimensionnement, la conversion de format, et l'assurance qualité des annotations.
- Entraîner et évaluer un modèle YOLOv8 pour la détection de cartons.

- Construire une application Streamlit complète qui permet aux utilisateurs de :
 - Téléverser des images de cartons empilés.
 - Recevoir une analyse des agencements de cartons, incluant le nombre de détections, les problèmes de surplomb, la stabilité des piles, et l'occupation des zones.
 - Visualiser l'historique des données d'analyse et configurer les paramètres du système.
 - Obtenir un score global de conformité pour la scène analysée.

1.3. Périmètre des Travaux

Le projet comprenait les éléments suivants :

- Acquisition et prétraitement du jeu de données SCD.
- Entraînement et ajustement fin d'un modèle de détection d'objets YOLOv8.
- Développement d'algorithmes pour l'analyse post-détection (surplomb, empilement, zones).
- Création d'une application web pour l'interaction utilisateur et la visualisation des résultats.
- Gestion de l'historique des analyses et des paramètres utilisateur.

2. Jeu de Données et Prétraitement

2.1. Description du Jeu de Données

Le principal jeu de données utilisé pour ce projet était le **SCD (Stacked Carton Dataset)**. Ce jeu de données est spécifiquement conçu pour les tâches impliquant la détection et la segmentation de cartons empilés dans divers agencements, le rendant adapté aux objectifs du projet. Le jeu de données original était fourni au format d'annotation COCO (Common Objects in Context). Pour ce projet, seule la classe "carton" était pertinente.



image 1: ici une image d'exemple du jeu de données SCD original, montrant des cartons empilés.

2.2. Pipeline de Prétraitement des Données

Un pipeline de prétraitement en plusieurs étapes a été mis en œuvre pour préparer le jeu de données SCD à l'entraînement du modèle YOLOv8. Cela comprenait le redimensionnement des images, la conversion de format, le nettoyage des annotations, et une visualisation approfondie.

• 2.2.1. Redimensionnement des Images et Annotations (resize.py)

- **Objectif** : Standardiser les dimensions des images d'entrée et ajuster les annotations correspondantes. Cette étape aide à gérer les ressources de calcul et peut influencer les performances du modèle.
- **Processus** : Le script `resize.py` a été utilisé pour redimensionner les images du jeu de données SCD et leurs annotations COCO correspondantes à une taille cible (par exemple, 800x800 pixels). Le script a recalculé les coordonnées des boîtes englobantes et les données de segmentation pour correspondre aux nouvelles dimensions de l'image. Cela a été effectué pour les ensembles d'entraînement et de validation.



Image 2 : une image comparative avant/après redimensionnement d'une image du jeu de données .

- **2.2.2. Conversion de Format (COCO vers YOLO)**

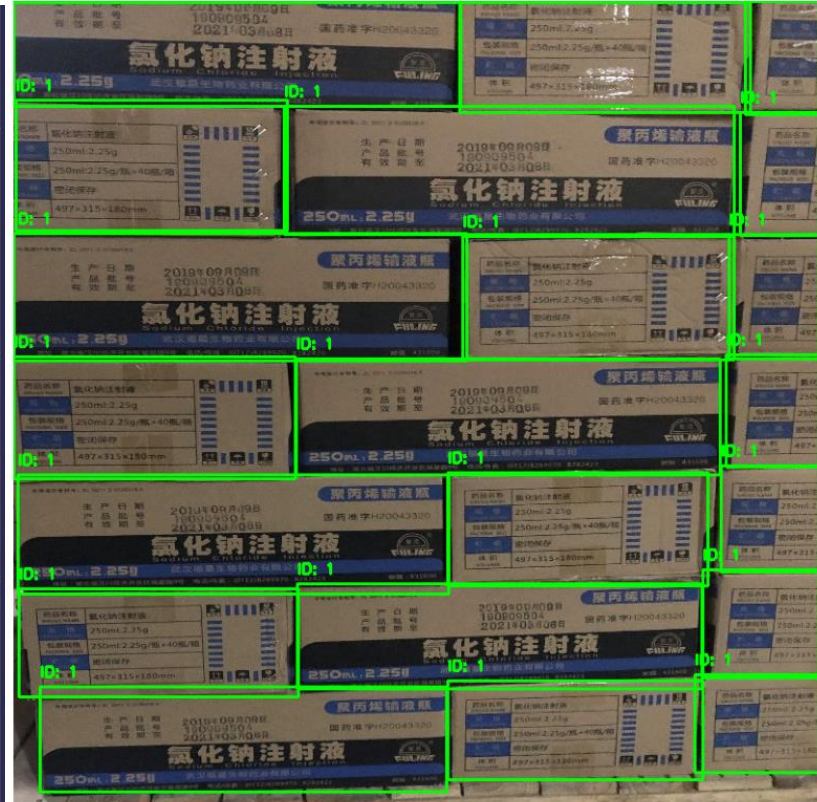
- **Objectif** : Le modèle YOLOv8 attend les annotations dans un format de fichier texte YOLO spécifique (class_id, x_center_norm, y_center_norm, width_norm, height_norm) plutôt que le format JSON COCO.
- **Processus** : Les annotations COCO redimensionnées ont été converties au format YOLO. Cela impliquait de normaliser les coordonnées des boîtes englobantes par rapport aux dimensions de l'image et de créer des fichiers .txt individuels pour chaque image.

- **2.2.3. Nettoyage des Annotations (fix.py)**

- **Objectif** : Les conversions initiales ou les annotations du jeu de données original contiennent parfois des erreurs telles que des boîtes englobantes avec des dimensions nulles/négatives ou des boîtes s'étendant au-delà des limites de l'image. De tels problèmes peuvent avoir un impact négatif sur l'entraînement du modèle.
- **Processus** : Le script fix.py (en particulier sa fonction sanitize_bboxes) a été utilisé pour nettoyer les annotations. Il a effectué des vérifications pour :
 - Les boîtes englobantes en dehors des limites de l'image, en les rognant aux limites de l'image.
 - Les boîtes englobantes avec de très petites surfaces (par exemple, MIN_BOX_AREA = 10 pixels) ou une largeur/hauteur non positive, en supprimant ces annotations invalides.
- **Impact** : Cette étape a considérablement amélioré la qualité et la fiabilité des données d'entraînement en supprimant ou corrigeant les étiquettes erronées.



Avant :



Après :

Image 3 : un exemple concret de la sortie du script fix.py, montrant le nombre de boîtes supprimées/corrigées, de "mauvaise" annotation avant et "bonne" après nettoyage.

• 2.2.4. Visualisation et Vérification des Données (coco_vis.py, yolo_vis.py)

- **Objectif :** Inspecter visuellement et confirmer l'exactitude des annotations à différentes étapes du pipeline de prétraitement.
- **Processus :**
 - coco_vis.py : Utilisé pour visualiser les annotations COCO originales ou redimensionnées sur leurs images respectives. Cela a aidé à comprendre la qualité initiale du jeu de données.
 - yolo_vis.py : Utilisé pour visualiser les boîtes englobantes à partir des fichiers d'étiquettes au format YOLO final. Ceci était crucial pour vérifier l'exactitude de la conversion COCO vers YOLO et du processus de nettoyage.
- **Observations :** Ces scripts de visualisation ont été essentiels pour identifier et corriger les problèmes dans le pipeline d'annotation, garantissant que le modèle a été entraîné sur des étiquettes de haute qualité.

3. Développement et Entraînement du Modèle

3.1. Sélection du Modèle

Le modèle de détection d'objets **YOLOv8** (You Only Look Once version 8) a été sélectionné pour ce projet. Plus précisément, le modèle pré-entraîné yolov8m.pt (moyen) a été utilisé comme point de départ.

- **Justification** : YOLOv8 est un modèle de détection d'objets en temps réel à la pointe de la technologie, connu pour son excellent équilibre entre vitesse et précision. Son architecture est efficace et bien adaptée à la détection d'objets comme les cartons. La disponibilité de poids pré-entraînés sur de grands jeux de données comme COCO permet un apprentissage par transfert efficace.

3.2. Environnement d'Entraînement

- **Matériel** : L'entraînement a été principalement effectué à l'aide d'un GPU compatible CUDA Local RTX 3060 (par exemple, device='0' spécifié dans le script d'entraînement, indiquant une configuration à un seul GPU).
- **Logiciel** :
 - Python
 - PyTorch
 - Bibliothèque Ultralytics YOLO

3.3. Configuration de l'Entraînement (training_script.py)

Le script training_script.py a été développé pour gérer le processus d'entraînement avec des paramètres configurables :

- **Modèle** : yolov8m.pt (pré-entraîné)
- **Jeu de données** : Chemin vers le fichier data.yaml définissant les chemins d'entraînement/validation et les noms de classes.
- **Époques** : 30.
- **Taille de lot (Batch Size)** : 8.
- **Taille d'image** : 640x640 pixels (imgsz=640).
- **Optimiseur** : AdamW.
- **Taux d'apprentissage** : LR initial de 0.01, avec un planificateur LR cosinus.

- **Augmentation des données** : Des techniques d'augmentation avancées ont été activées (augment=True), incluant des ajustements HSV, rotation, translation, mise à l'échelle, cisaillement, changements de perspective, retournements, mosaïque, mixup, et copier-coller. Celles-ci aident à améliorer la généralisation et la robustesse du modèle.
- **Autres Paramètres Clés** : Seuil de confiance (0.001 pour l'entraînement, plus élevé pour l'inférence), seuil IoU (0.6 pour NMS pendant l'entraînement), label_smoothing, époques close_mosaic.

3.4. Processus d'Entraînement

Le modèle a été entraîné à l'aide du script training_script.py. Le script gère le chargement du modèle, du jeu de données, l'application des augmentations, et l'exécution de la boucle d'entraînement. La validation a été effectuée périodiquement pendant l'entraînement pour surveiller les performances sur des données non vues et sauvegarder le point de contrôle du modèle le plus performant (best.pt).

3.5. Évaluation des Performances

Les performances du modèle ont été évaluées sur l'ensemble de validation à l'aide de métriques standard de détection d'objets. Les métriques de validation finales obtenues étaient :

- **Précision** : 0.954 (95.4% des boîtes prédites étaient correctes)
- **Rappel** : 0.936 (93.6% des boîtes réelles ont été détectées)
- **mAP@0.50 (mAP50)** : 0.975 (Performance exceptionnelle à un seuil IoU de 0.5)
- **mAP@0.50-0.95 (mAP50-95)** : 0.862 (Forte performance sur une plage de seuils IoU)

Ces résultats indiquent que le modèle YOLOv8 entraîné fonctionne très bien pour détecter avec précision les cartons. Des tests d'inférence initiaux ont également été menés à l'aide de scripts comme test_inference.py pour évaluer rapidement les prédictions du modèle sur des images d'exemple.

4. Application : Analyseur de Conformité des Cartons (app.py)

4.1. Vue d'ensemble

Une application web Streamlit complète, nommée "Analyseur de Conformité des Cartons," a été développée pour fournir une interface utilisateur pour le modèle entraîné et la logique d'analyse associée. Cette application permet aux utilisateurs d'exploiter la puissance du modèle de détection de cartons pour des vérifications pratiques de conformité.

- **Pile Technologique** : Streamlit (pour l'UI), Python, Ultralytics YOLO (pour l'inférence), OpenCV (pour le traitement d'image), Plotly (pour les graphiques), PIL (pour la manipulation d'images).

4.2. Fonctionnalités Clés

• 4.2.1. Tableau de Bord :

- Fournit un aperçu de l'utilisation du système et des tendances de performance.
- Affiche des métriques telles que le nombre total d'analyses, le score de conformité moyen, et le nombre d'exécutions à faible conformité.
- Visualise les tendances pour la conformité, les détections, et les temps de traitement à l'aide de graphiques Plotly.

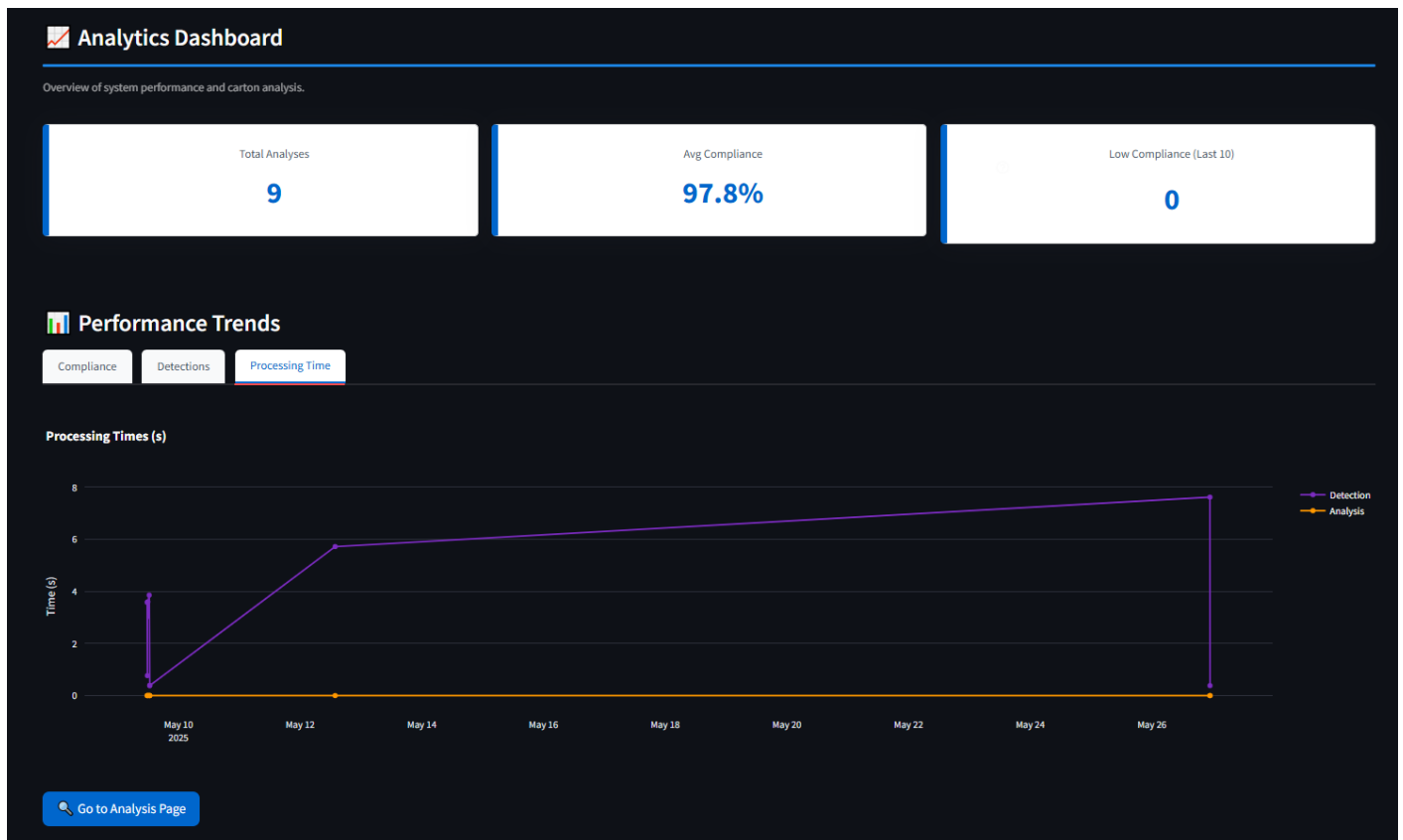


Image 4 :Une capture d'écran du tableau de bord de l'application Streamlit.

• 4.2.2. Analyse d'Image :

- **Téléversement d'Image** : Les utilisateurs peuvent téléverser des images (JPG, PNG, JPEG) contenant des cartons empilés.
- **Détection de Cartons** : L'image téléversée est traitée par le modèle YOLOv8 entraîné (best.pt) pour détecter tous les cartons.
- **Analyse des Surplombs entre Paires** : Détecte les cas où un carton surplombe un autre carton en dessous au-delà d'un seuil configurable.

- **Analyse Avancée des Piles** : Identifie les piles verticales de cartons et les analyse pour :
 - **Conformité de la Hauteur** : Vérifie si les piles dépassent une hauteur maximale autorisée.
 - **Qualité de l'Alignement** : Évalue l'alignement horizontal des cartons au sein d'une pile.
- **Comptage par Zone** : Permet aux utilisateurs de définir des zones rectangulaires personnalisées sur l'image et compte le nombre de cartons dans chaque zone.
- **Score de Conformité** : Calcule un score global de conformité (0-100%) basé sur les problèmes détectés (surplombs, problèmes de pile, non-concordance des comptes, cartons hors zone).
- **Sortie Visualisée** : L'image traitée est affichée avec des boîtes englobantes autour des cartons détectés, des mises en évidence pour les surplus, les connexions de piles, et les zones définies.

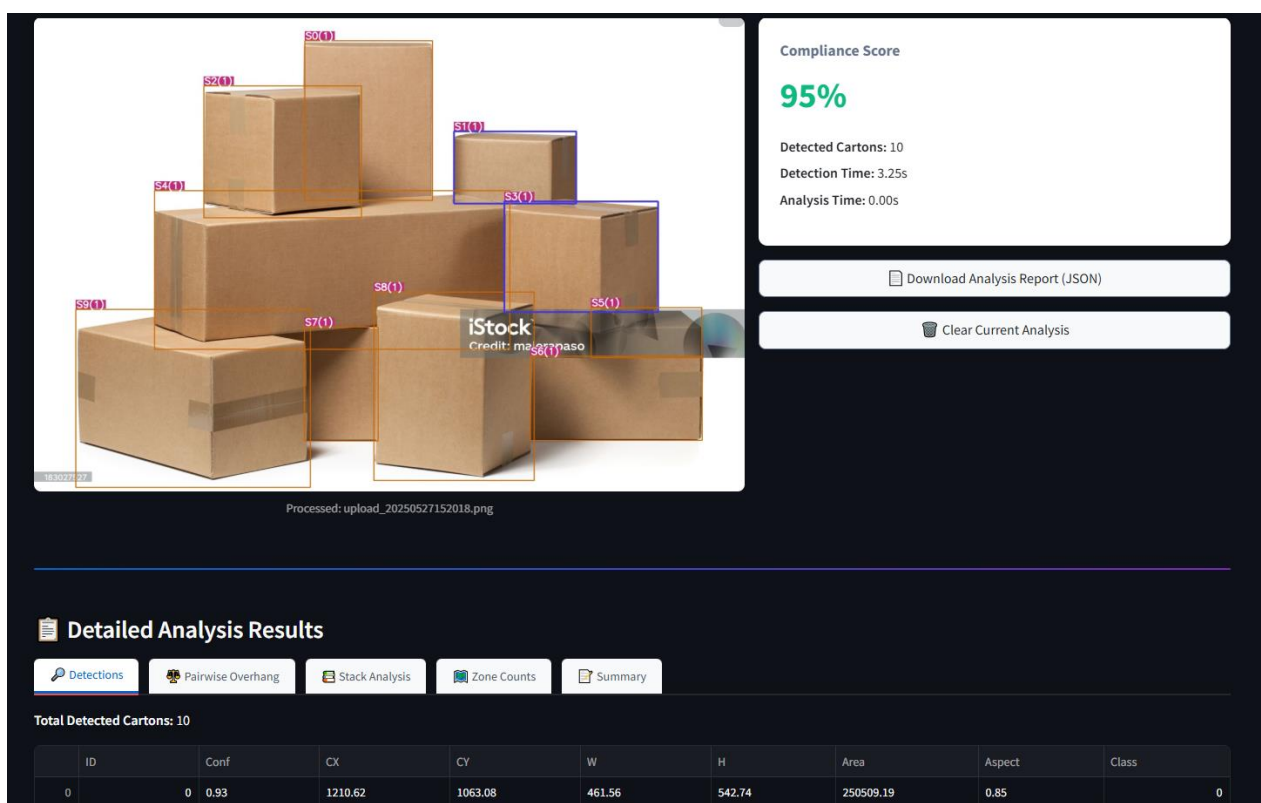


image 5: Une capture d'écran de la page d'analyse de l'application montrant une image téléversée avec les détections, les annotations de surplus, les piles identifiées et les zones (si configurées).

- **4.2.3. Historique des Analyses :**

- Stocke les résultats des analyses précédentes dans un fichier JSON (analysis_history.json).
- Permet aux utilisateurs de sélectionner et de revoir les résultats détaillés des analyses passées.
- Option pour supprimer des entrées d'historique individuelles ou effacer tout l'historique.

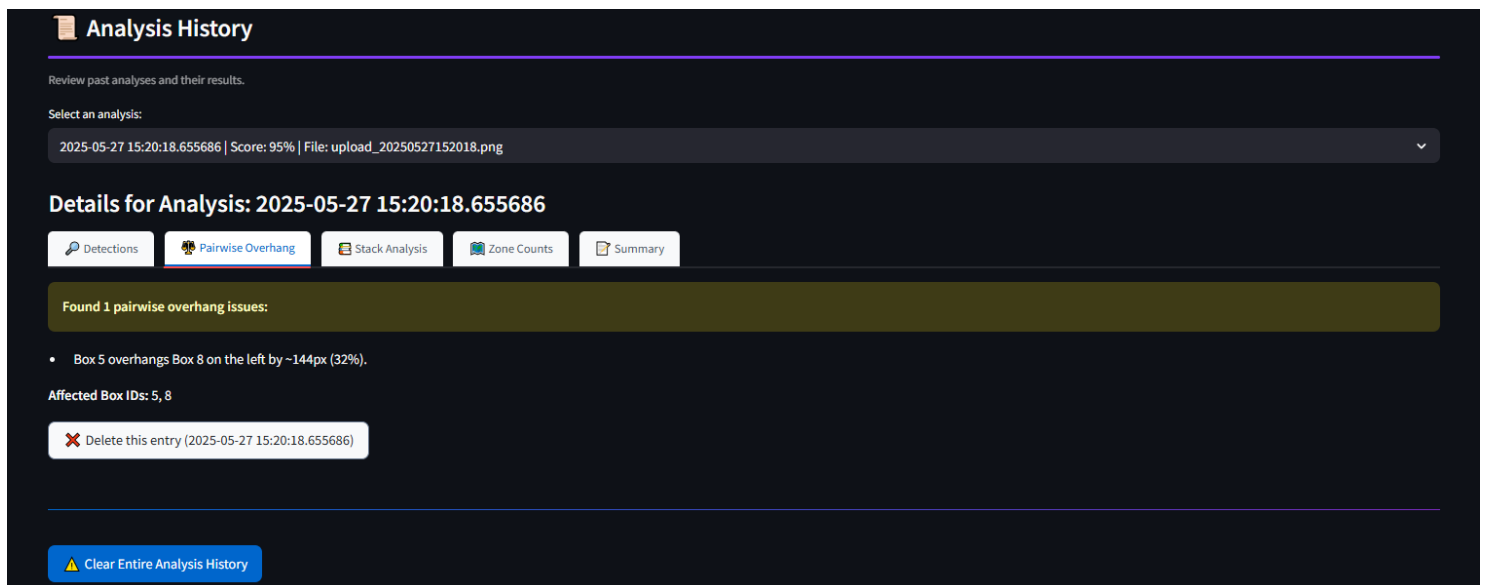


Image 5 : Une capture d'écran de la page d'historique, montrant la liste des analyses passées.

- **4.2.4. Configuration du Système:**

- **Paramètres de Détection :** Les utilisateurs peuvent ajuster les seuils de confiance et d'IoU pour le modèle YOLO.
- **Modules d'Analyse :** Activer/désactiver et configurer les paramètres pour l'analyse des surplombs, l'analyse des piles, et le comptage par zone (par exemple, ratio de surplomb max, hauteur de pile max, coordonnées et couleurs des zones).
- **Gestion des Données :** Sauvegarder les paramètres actuels dans un fichier (user_settings.json), charger les paramètres sauvegardés, ou réinitialiser aux valeurs par défaut.

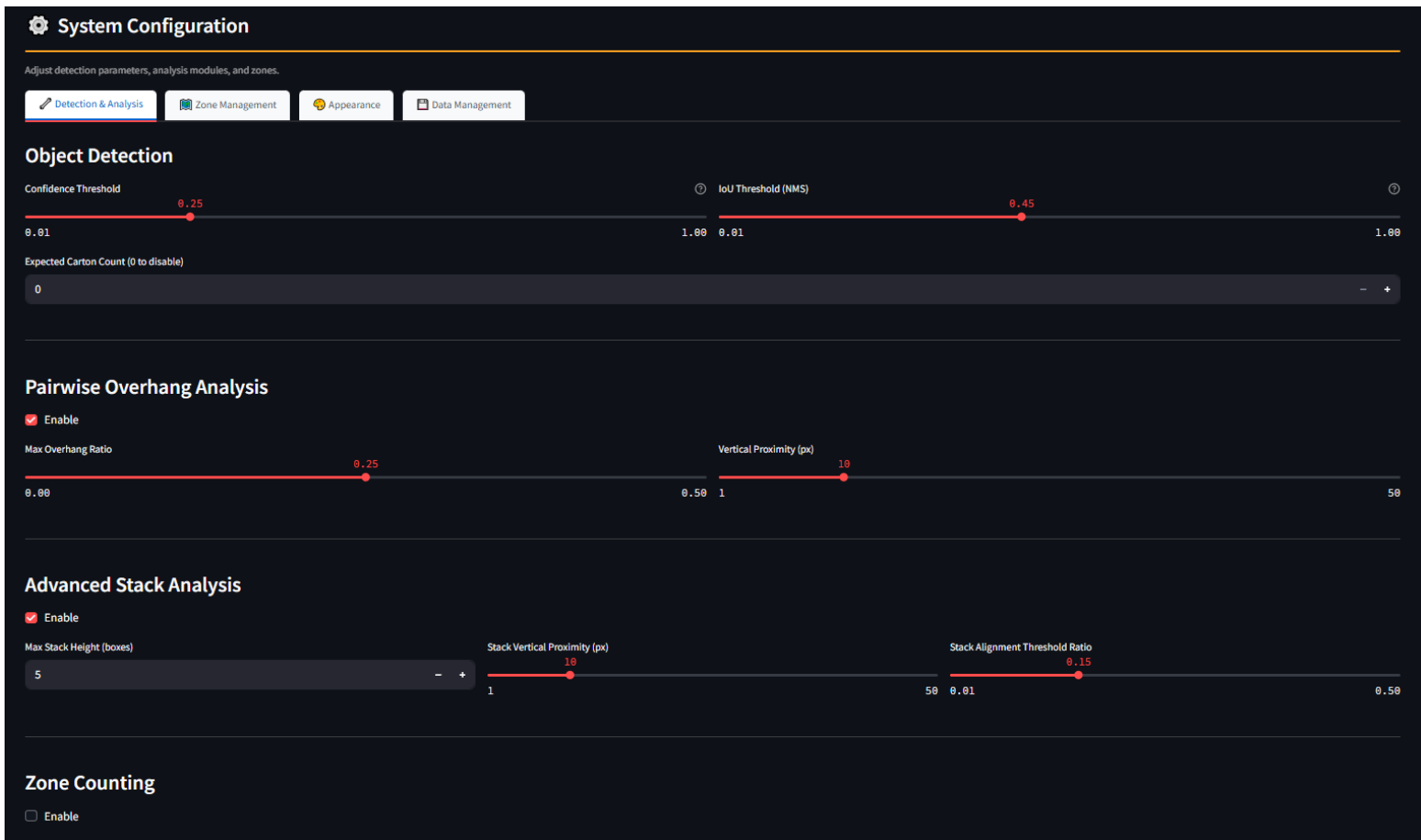


Image 6 : Une capture d'écran de la page des paramètres, montrant les options de configuration.

- **4.2.5. Interface Utilisateur :**

- Une interface propre et intuitive construite avec Streamlit.
- Navigation par barre latérale pour les différentes pages (Tableau de Bord, Analyse, Historique, Paramètres).
- CSS personnalisé (load_custom_css) pour une thématisation visuelle améliorée et le style des composants comme les cartes de métriques, les en-têtes, et les onglets.

4.3. Architecture Système (Haut Niveau)

- **Frontend :** Interface utilisateur basée sur Streamlit s'exécutant dans un navigateur web.
- **Backend :** Logique applicative Python gérant :
 - Le traitement d'image et l'inférence du modèle YOLOv8.
 - Les algorithmes d'analyse de conformité.
 - La gestion de l'état de session.
 - L'interaction avec les fichiers de données.

- **Stockage des Données :**

- analysis_history.json : Stocke les enregistrements détaillés des analyses passées.
- user_settings.json : Stocke les paramètres de l'application configurés par l'utilisateur.
- weights/best.pt : Le fichier du modèle YOLOv8 entraîné.

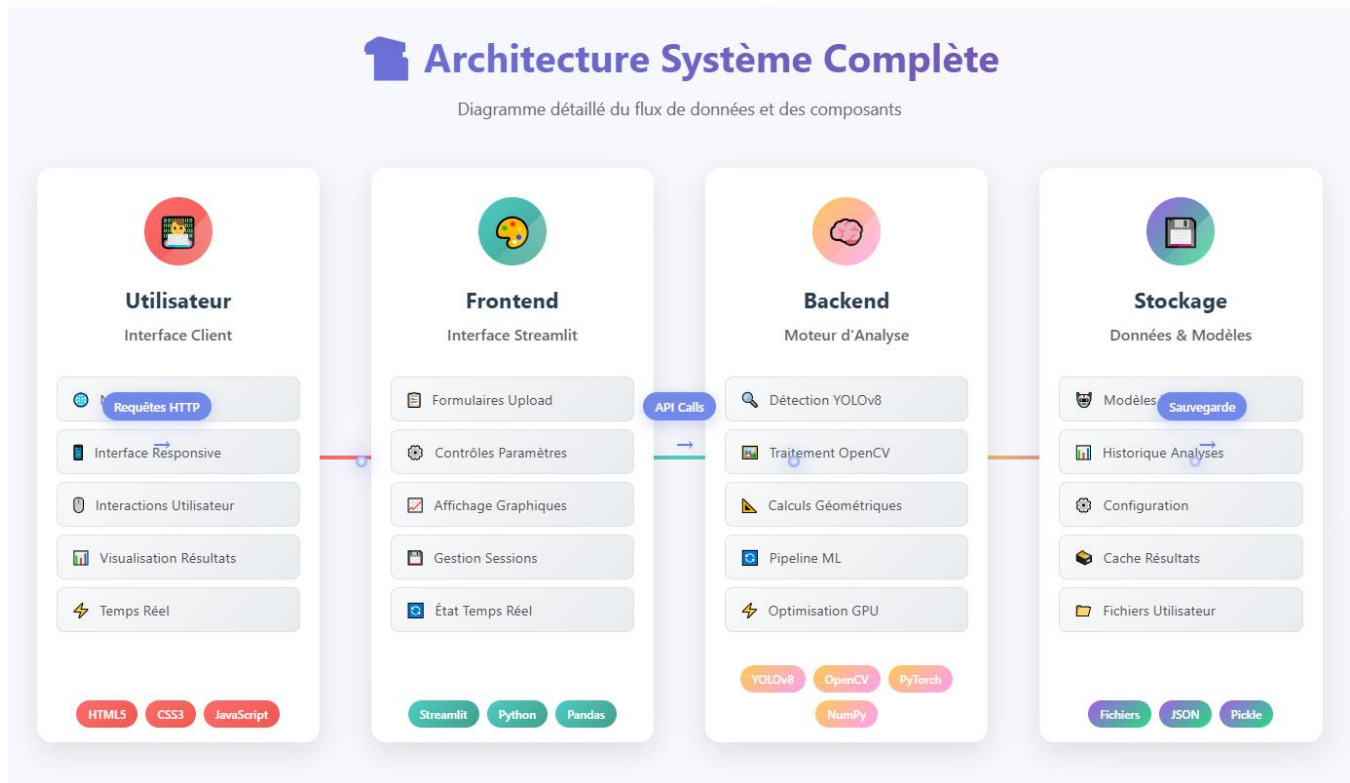


Image 7 : Un schéma simple de l'architecture système

5. Défis et Solutions

- **Qualité du Jeu de Données :** Le jeu de données SCD initial, même après conversion, présentait quelques défis d'annotation (par exemple, des boîtes englobantes incorrectes).
 - **Solution :** Mise en œuvre d'un script de nettoyage des annotations (convert.py) pour nettoyer les boîtes englobantes en les rognant, en supprimant celles invalides, et en assurant une surface minimale. Une visualisation extensive (coco_vis.py, yolo_vis.py) a été utilisée avant et après les étapes de traitement pour confirmer la qualité des étiquettes.

- **Généralisation du Modèle** : S'assurer que le modèle fonctionne bien sur divers types de cartons, conditions d'éclairage, et motifs d'empilement.
 - **Solution** : Exploitation de l'architecture robuste de YOLOv8 et de techniques d'augmentation de données étendues pendant l'entraînement (tel que configuré dans `improved_training_script.py`) pour améliorer la généralisation.
- **Définition des Règles de Conformité** : Traduire les règles de conformité qualitatives (par exemple, "pile stable") en métriques quantifiables pour une analyse automatisée.
 - **Solution** : Développement d'algorithmes spécifiques dans `app.py` pour la détection des surplombs (basée sur le pourcentage et la proximité), la hauteur des piles (nombre max de boîtes), et l'alignement des piles (ratio de déviation centrale). Ces paramètres sont configurables par l'utilisateur.

6. Conclusion et Travaux Futurs

6.1. Résumé des Réalisations

Ce projet a développé avec succès un Analyseur de Conformité des Cartons automatisé. Un modèle YOLOv8 robuste a été entraîné, atteignant une haute précision (mAP@50 de 0.975) pour la détection de cartons sur le jeu de données SCD traité. L'application Streamlit associée fournit une interface puissante et conviviale pour analyser les agencements de cartons, offrant des fonctionnalités telles que la détection des surplombs, l'analyse des piles, le comptage par zone, et un score de conformité complet. Le système démontre le potentiel de l'apprentissage profond pour automatiser et améliorer les processus d'audit en entrepôt.

6.2. Travaux Futurs

Plusieurs pistes existent pour des améliorations futures :

- **Analyse en Temps Réel** : Adapter le système pour l'analyse de flux vidéo en temps réel à partir de caméras d'entrepôt.
- **Intégration avec WMS** : Intégrer l'analyseur avec les Systèmes de Gestion d'Entrepôt (WMS) existants pour un flux de données transparent et des alertes automatisées.
- **Informations de Segmentation** : Utiliser les capacités de segmentation (si le modèle est entraîné pour cela ou si les masques de segmentation SCD sont utilisés) pour des calculs de surface plus précis ou la détection de défauts sur les cartons.
- **Analyses Avancées** : Incorporer des analyses plus sophistiquées, telles que la prédiction de la probabilité d'effondrement des piles ou l'optimisation de la densité de stockage basée sur la conformité.

- **Détection de Défauts Étendue** : Entraîner le modèle à identifier d'autres types de défauts de cartons (par exemple, boîtes endommagées, étiquettes incorrectes) au-delà de la conformité d'empilement.
- **Application Mobile** : Développer une version mobile de l'application pour les inspections en déplacement par le personnel d'entrepôt.
- **Compréhension 3D Améliorée** : Explorer des techniques pour inférer les relations 3D entre les cartons pour une analyse plus précise des surplombs et de la stabilité, éventuellement en utilisant des caméras de profondeur ou la stéréovision multi-vues.



Explorez notre
Projet sur GitHub
(QR)